BENEMÉRITA UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS
Departamento de Ciencias Computacionales

Actividad de Aprendizaje 8. Programa 4. Algoritmo de Planificación FCFS (First-Come-First-Served) — Continuación

2026-B
Sección D09
Sistemas Operativos
Código de la materia: IL366

P R E S E N T A (N)
ICOM - Velázquez Vazquéz Valeria - 224785718

PROFESOR(A)
Becerra Velázquez Violeta del Rocío

Guadalajara, Jalisco, Mx.
18 de Octubre de 2026

Índice

Objetivo de la actividad 3
Notas acerca del lenguaje 3
Arquitectura y estructuras de datos 3
Reto principal y cómo se solucionó 4
Capturas de pantalla 5
Conclusiones 6
Bibliografía 6

Tabla de Imágenes

Ilustración 1. Barra de captura de teclado estilo vim, sin uso de Enter 5
Ilustración 2. Tabla de procesos (BCP) desplegada con la tecla B 5

Objetivo de la Actividad

Dar continuidad al simulador de planificación FCFS (First-Come-First-Served) desarrollado en la Actividad 6, incorporando una interacción de teclado en tiempo real: el usuario ya no escribe la letra del comando en un campo de texto y presiona Enter, sino que el programa reconoce la tecla en el instante en que se presiona, de forma similar al modo de comandos del editor Vim. Asimismo, se completó la cobertura de los requerimientos de la actividad al incorporar las dos interrupciones que faltaban por programar: la tecla "N", que genera un proceso nuevo bajo demanda, y la tecla "B", que pausa la simulación y despliega la Tabla de Control de Procesos (BCP) con el estado actual de todos los procesos del sistema.

En conjunto, el sistema debe seguir administrando una memoria restringida a un grado de multiprogramación máximo de cinco procesos, atendiendo la cola de listos bajo el algoritmo FCFS, controlando las interrupciones por entrada/salida y los errores, y calculando con precisión los tiempos de planificación (llegada, respuesta, espera, servicio, retorno y finalización) tanto de forma instantánea, a través de la tecla B, como en el reporte final que se despliega al concluir la ejecución de todos los procesos.

Notas acerca del Lenguaje

Se continuó utilizando Python junto con la biblioteca gráfica PySide6 (Qt), ya que su modelo orientado a eventos y su clase QTimer siguen siendo ideales para emular los ciclos de reloj del procesador sin recurrir a hilos adicionales. El cambio relevante en esta entrega está en el manejo del teclado: en lugar de depender de un QLineEdit y de la señal returnPressed, la ventana principal (QMainWindow) sobrescribe directamente el método keyPressEvent. Para que esto funcione de manera confiable, se retiró el foco de todos los widgets que no lo necesitan (las tablas ya usaban setFocusPolicy(Qt.NoFocus) desde la actividad anterior) y se fijó un foco fuerte (Qt.StrongFocus) sobre la ventana principal, de modo que cualquier tecla presionada llegue directamente al controlador de comandos sin que ningún otro control la intercepte.

Para dar retroalimentación inmediata al usuario sobre la tecla registrada, sin interrumpir la simulación con cuadros de diálogo emergentes en cada pulsación, se agregó una pequeña barra de estado con tipografía monoespaciada y fondo negro, inspirada en la línea de comandos de Vim, que muestra brevemente la tecla reconocida (en verde) o el aviso de tecla no válida (en rojo) y vuelve a su estado de reposo mediante un QTimer de un solo disparo.

Arquitectura y Estructuras de Datos

Se conservó el patrón Modelo-Vista-Controlador utilizado en la actividad previa. El Bloque de Control de Procesos (PCB) sigue representado por la clase Process, decorada con @dataclass, que concentra el identificador, la operación, el Tiempo Máximo Estimado (TME) y las variables de planificación (llegada, finalización, respuesta y tiempo bloqueado), además de las propiedades derivadas (tiempo restante, tiempo de retorno, tiempo de espera y estado final). Las cuatro colas del sistema (new_queue, ready_queue, blocked_queue y terminated_queue) continúan implementadas como listas dinámicas FIFO dentro de la clase SimulatorEngine.

Sobre esa base se incorporaron dos capacidades nuevas en el motor de simulación. La primera es el método request_new_process(), asociado a la tecla "N": genera un proceso con datos aleatorios, lo agrega a la cola de Nuevos y delega en el planificador a largo plazo (_admit_new_processes) la decisión de admitirlo de inmediato a memoria o dejarlo en espera si ya se alcanzó el máximo de cinco procesos concurrentes. La segunda es la construcción, bajo demanda, de una fotografía (snapshot) del sistema completo para la tecla "B": se concatenan los procesos de las cinco colas (Nuevo, Listo, Ejecución, Bloqueado y Terminado) y se envían a una ventana de solo lectura que calcula, para cada proceso, el campo correspondiente según su estado —por ejemplo, el tiempo restante de bloqueo para los procesos en E/S, o si el proceso terminó de forma normal o por error— sin alterar el estado interno del motor.

Reto Principal y Cómo se Solucionó

El mayor reto de esta entrega fue coordinar la pausa que exige la tecla "B" con la pausa ya existente de la tecla "P", sin que ambas interfirieran entre sí. Si la simulación ya estaba en pausa por "P" y el usuario presionaba "B" para consultar la tabla de procesos, cerrar esa ventana con "C" no debía reanudar automáticamente el reloj, porque la pausa original seguía vigente. La solución fue registrar, justo antes de mostrar la tabla, si la simulación ya estaba pausada; al cerrarse la ventana emergente, el programa solo reanuda el QTimer y el motor si esa bandera indica que la pausa fue provocada únicamente por la tecla "B"; en caso contrario, la simulación permanece pausada tal como la dejó el usuario.

Un segundo reto fue calcular el "tiempo de espera al momento" que pide el requerimiento para la tecla "B", ya que las propiedades existentes en la clase Process (como waiting_time) sólo están definidas una vez que el proceso terminó. Para los procesos que aún siguen en el sistema se adoptó la misma definición que usa el reporte final —tiempo de retorno menos tiempo de servicio— pero evaluada en vivo: se resta al tiempo transcurrido desde la llegada del proceso el tiempo que ya alcanzó a ejecutarse, de modo que la cifra mostrada en la tabla emergente sea consistente con la que aparecerá en el BCP final una vez que el proceso concluya.

Finalmente, fue necesario asegurar que la captura directa del teclado no rompiera la barra de progreso, ya que ahora el número total de procesos puede crecer en cualquier momento por la tecla "N". Esto se resolvió llevando un contador total_generated dentro del motor de simulación, que se incrementa cada vez que se crea un proceso nuevo y se usa como denominador al calcular el porcentaje de avance, en lugar del número fijo de procesos con el que arrancó el programa.

Capturas de Pantalla

Las siguientes ilustraciones corresponden a la ejecución del programa con las teclas E, W, P, C, N y B ya integradas a la captura directa de teclado.

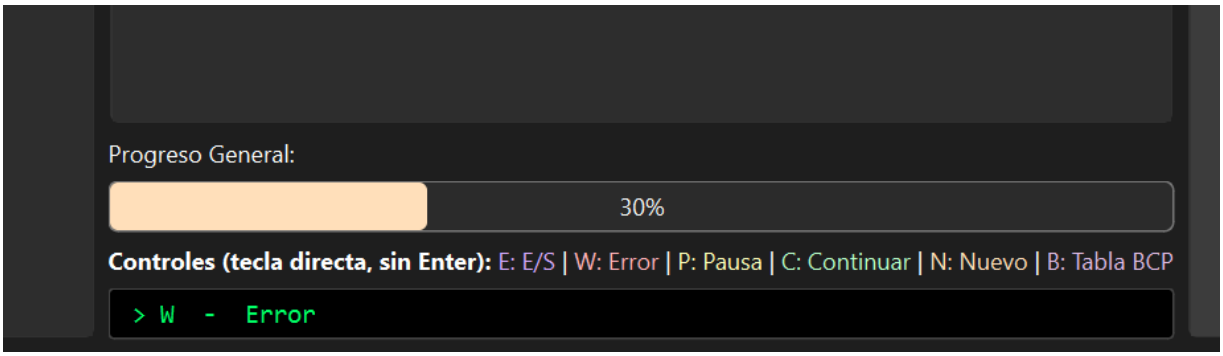
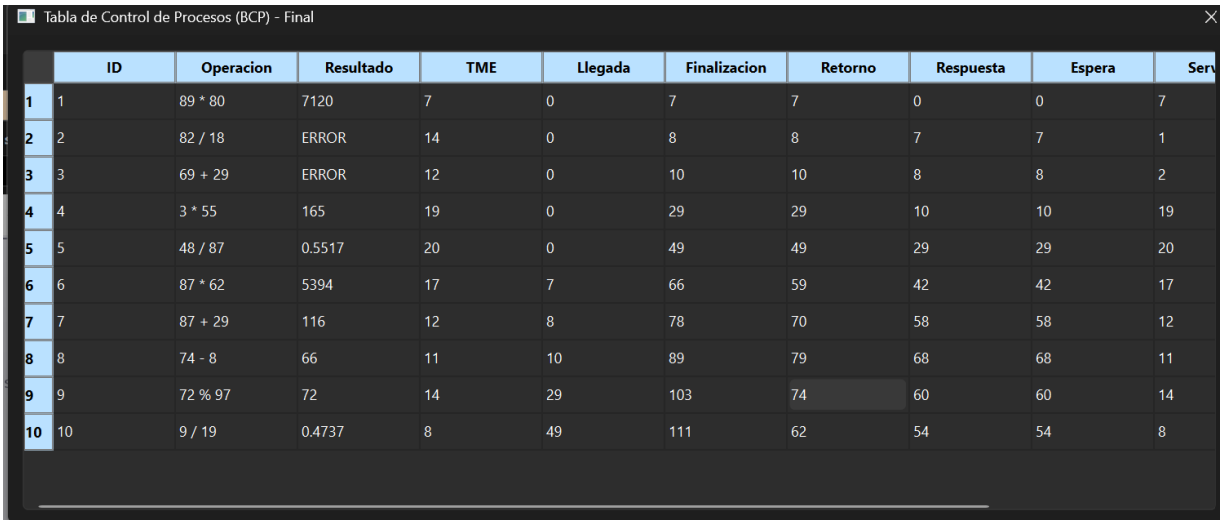


Ilustración 1. Barra de captura de teclado



	ID	Operacion	Resultado	TME	Llegada	Finalizacion	Retorno	Respuesta	Espera	Serv
1	1	89 * 80	7120	7	0	7	7	0	0	7
2	2	82 / 18	ERROR	14	0	8	8	7	7	1
3	3	69 + 29	ERROR	12	0	10	10	8	8	2
4	4	3 * 55	165	19	0	29	29	10	10	19
5	5	48 / 87	0.5517	20	0	49	49	29	29	20
6	6	87 * 62	5394	17	7	66	59	42	42	17
7	7	87 + 29	116	12	8	78	70	58	58	12
8	8	74 - 8	66	11	10	89	79	68	68	11
9	9	72 % 97	72	14	29	103	74	60	60	14
10	10	9 / 19	0.4737	8	49	111	62	54	54	8

Ilustración 2. Tabla de procesos (BCP) desplegada con la tecla B

Conclusiones

Migrar la entrada de comandos de un QLineEdit con Enter a una captura directa de teclado permitió comprender mejor cómo Qt distribuye los eventos de entrada entre los distintos widgets de una ventana, y por qué es indispensable retirar el foco de los controles que no lo necesitan para que un evento como keyPressEvent llegue a donde se espera. Añadir las teclas "N" y "B" también reforzó la separación entre el motor de simulación y la interfaz: toda la lógica de admisión de procesos y de cálculo de tiempos permanece en SimulatorEngine y en Process, mientras que la ventana principal únicamente traduce la tecla presionada en una llamada al modelo y refresca la pantalla.

En conjunto, esta segunda iteración del simulador deja una base más completa para observar el comportamiento del planificador FCFS: ahora es posible inspeccionar el Bloque de Control de cualquier proceso en cualquier momento de la ejecución, no solo al finalizar, lo que hace más evidente el efecto convoy y el costo de mantener procesos bloqueados o en espera cuando la memoria está saturada.

Bibliografía

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10a ed.). Wiley.

The Qt Company. (2024). PySide6 Reference Documentation — QMainWindow, QKeyEvent y QTimer. Recuperado de la documentación oficial de Qt for Python.

Moolenaar, B. (s. f.). Vim Documentation — Command-line mode. Recuperado de la documentación oficial del proyecto Vim.